kay.heider [☺] tu-dortmund.de

Exercise for lecture Real-Time Operating Systems Design and Implementation
Winter semester 2025/2026

Exercise Sheet 3

Thursday, January 8, 2026

3.1 Interrupts

Take your previous implementation of the stopwatch. You should have a task, which periodically reads the state of a button to check if the button is pressed. This mechanism is called “polling” and can be replaced by interrupts.
First, the header `<driver/gpio.h>` has to be included to configure interrupts by GPIO pins.
In order to activate an interrupt, which is triggered by a button press, you will have to create an interrupt handler function similar to the following scheme:

```
static void handler(void *args)
```

Next, you will have to enable interrupts by GPIO pins at runtime with the command:

```
gpio_install_isr_service(0);
```

Afterwards, you can configure when an interrupt should be triggered by a specific GPIO pin, for example, to configure that an interrupt will be triggered when a depressed button is released, you can call:

```
gpio_set_intr_type(<gpio_num>, GPIO_INTR_NEGEDGE);
```

Finally, you can assign your previously created handler function to an interrupt source with:

```
gpio_isr_handler_add(<gpio_num>, handler, <arg>);
```

The GPIO pin numbers of the buttons are given by the macros `TOP_LEFT`, `BOTTOM_LEFT`, ...

a. Realize the interrupt mechanism for all required buttons in your current implementation.

b. Apply the deferred interrupt scheme to properly communicate between interrupt handlers and the other tasks.

3.2 Performance Optimization

Please copy the files from the template folder to the “main” folder. In this template, a periodic task executes inference of a decision tree, which is a lightweight machine learning model. The task is strictly periodic and releases a job every 1000th tick. Unfortunately, the inference of the decision tree takes 1021 ticks, making the system infeasible.

a. Go through the implementation of the decision tree workload (`dt.hpp`). Understand the implementation.

b. Identify the deadline misses of the task, that calls the unoptimized version.

c. Implement an alternative function `predict_optimized`, where you try to make the execution of the decision tree faster without changing the output.

d. For your optimized implementation, compare the resulting accuracy to the not optimized version. They must be the same.

e. Can you make the system feasible, with only optimizing the implementation?